

1. Explain the term Race condition in thread synchronization.
2. Explain mutex and how mutex helps to avoid race condition.
3. Write a C program that creates n threads using `argv[1]`. Each thread should print its thread ID along with a message such as "Hello from thread X". The main thread must wait for all threads to finish using `pthread_join()`.
4. Write a program where each thread receives a unique integer (for example, its index). Each thread should print the number it received and determine whether it is even or odd. Use a struct or another safe method to pass arguments to threads.
5. Create a program where each thread generates a random number between 1 and 1000 and returns it using `pthread_exit()`. The main thread should collect all returned values, print them, and determine the maximum value among them.
6. Write a program where multiple threads increment a shared global counter (e.g., 1,000,000 times each). First run the program without synchronization, then modify it to use a mutex (`pthread_mutex_t`). Compare and print the results.
7. Write a program that takes a number N from `argv[1]` and uses n threads to compute the sum of numbers from 1 to N . Divide the work evenly among threads (slicing). Each thread should return its partial sum, and the main thread should combine the results into the final sum.

The End